REST の実装の詳細

ONTAP Automation

NetApp
February 17, 2022

This PDF was generated from https://docs.netapp.com/ja-jp/ontap-automation/rest_web_services_foundation.html on February 17, 2022. Always check docs.netapp.com for the latest.

目次

REST の実装の詳細 1
基盤としての REST Web サービス 1
ONTAP REST API のバージョン管理 2
ONTAP API へのアクセス方法 2
基本的な動作特性 3
API 要求を制御する入力変数 5
API 応答の解釈 8
ジョブオブジェクトを使用した非同期処理の動作 10

REST の実装の詳細

REST では共通のテクノロジとベストプラクティスが確立されますが、各 API の詳細は開発時の選択内容によって異なる場合があります。ライブ環境で使用する前に、 ONTAP REST API の設計上の特徴を理解しておく必要があります。

基盤としての REST Web サービス

Representational State Transfer （ REST ）は、分散 Web アプリケーションの作成に使用される形式です。Web サービス API の設計においては、サーバベースのリソースを公開してその状態を管理するための一連のテクノロジとベストプラクティスが確立されます。主流のプロトコルと標準が使用されており、 ONTAP クラスタを管理するための柔軟な基盤が提供されます。

リソースと状態の表示

リソースは、 Web ベースシステムの基本コンポーネントです。REST Web サービスアプリケーションを作成する場合、設計の早い段階で次の作業を行います。

- システムまたはサーバベースのリソースの識別

  すべてのシステムは、リソースを使用および管理します。リソースには、ファイル、ビジネストランザクション、プロセス、管理エンティティなどがあります。REST Web サービスに基づいてアプリケーションを設計する際に行う最初の作業の 1 つは、リソースを識別することです。

- リソースの状態および関連する状態操作の定義

  リソースの状態の数は有限で、リソースは必ずそのいずれかの状態にあります。状態、および状態の変化に影響する関連操作を明確に定義する必要があります。

URI エンドポイント

すべての REST リソースは、明確に定義されたアドレス指定方式を使用して定義および使用可能にする必要があります。リソースが置かれているエンドポイントは、 Uniform Resource Identifier （ URI ）で識別されます。URI は、ネットワーク内の各リソースに一意の名前を作成するための一般的なフレームワークです。Uniform Resource Locator （ URL ）は、リソースを識別してアクセスするために Web サービスで使用される URI の一種です。リソースは通常、ファイルディレクトリに似た階層構造で公開されます。

HTTP メッセージ

Hypertext Transfer Protocol （ HTTP ）は、 Web サービスのクライアントとサーバがリソースに関する要求と応答のメッセージを交換する際に使用するプロトコルです。Web サービスアプリケーションの設計の一環として、 HTTP メソッドはリソースおよび対応する状態管理アクションにマッピングされます。HTTP はステートレスです。したがって、関連する一連の要求と応答を 1 つのトランザクションの一部として関連付けるには、要求と応答のデータフローで伝送される HTTP ヘッダーに追加情報を含める必要があります。

JSON 形式

Web サービスのクライアントとサーバの間で情報を構造化して転送する方法は複数ありますが、最も広く使用されているのは JavaScript Object Notation （ JSON ）です。JSON は、単純なデータ構造をプレーンテキストで表すための業界標準であり、リソースについての状態情報の転送に使用されます。ONTAP REST API では、 JSON を使用して、各 HTTP 要求と応答の本文で伝送されるデータをフォーマットします。

ONTAP REST API のバージョン管理

ONTAP 9 に付属の REST API には、 ONTAP のリリース番号とは関係のないバージョン番号が割り当てられています。使用している ONTAP のリリースに含まれている API のバージョンと、 API の使用にどのような影響があるかを確認しておく必要があります。

ONTAP 9 のすべてのバージョンで同じバージョンの REST API を使用します。

ONTAP 9.6 ～ ONTAP 9.10.1

バージョン 1 以降

"ONTAP 9 リリースノート"

ONTAP API へのアクセス方法

ONTAP REST API には、いくつかの方法でアクセスできます。

ネットワークに関する考慮事項

REST API には、次のインターフェイスを通じて接続できます。

- クラスタ管理 LIF
- ノード管理 LIF
- SVM 管理 LIF

使用する LIF は、 HTTPS 管理プロトコルをサポートするように設定する必要があります。また、ネットワークのファイアウォール設定で HTTPS トラフィックを許可する必要があります。

必ずクラスタ管理 LIF を使用してください。これにより、 API 要求の負荷がすべてのノードに分散され、オフラインになっているノードや接続の問題が発生しているノードは回避されます。

ONTAP API オンラインドキュメントページ

ONTAP API オンラインドキュメントページに、 Web ブラウザを使用する際のアクセスポイントが記載されています。個々の API 呼び出しを直接実行する方法に加え、各呼び出しの入力パラメータやその他のオプションなど、 API の詳細な概要も含まれています。API 呼び出しは機能のカテゴリ別に分類されています。を参照してください "REST リソースカテゴリの概要" を参照してください。

最新バージョンの API のドキュメントページにアクセスするための URL の形式は次のとおりです。

https://<cluster_mgmt_ip_address>/docs/api`

カスタムのソフトウェアとツール

ONTAP API には、さまざまなプログラミング言語やツールを使用してアクセスできます。代表的なものとしては、 Python 、 Java 、 Curl 、 PowerShell などがあります。API を使用するプログラム、スクリプト、またはツールは、 REST Web サービスのクライアントとして機能します。プログラミング言語を使用すると、API についての理解が深まり、 ONTAP の管理を自動化することもできます。

最新バージョンの API に直接アクセスするためのベース URL の形式は次のとおりです。

https://<cluster_mgmt_ip_address>/api`

複数のバージョンがサポートされている場合に特定のバージョンの API にアクセスするには、次の形式の URL を使用します。

https://<cluster_mgmt_ip_address>/api/v1`

基本的な動作特性

REST で共通のテクノロジとベストプラクティスは確立されますが、各 API の詳細は設計内容に応じて異なる場合があります。

要求と応答の **API** トランザクション

すべての REST API 呼び出しは、 ONTAP システムへの HTTP 要求として実行され、クライアントへの関連する応答が生成されます。この要求と応答のペアで API トランザクションが構成されます。API を使用する前に、要求の制御に使用できる入力変数と応答出力の内容を理解しておく必要があります。

CRUD 操作のサポート

ONTAP REST API で使用できる各リソースへのアクセスは CRUD モデルに基づきます。

- 作成
- 読み取り
- 更新
- 削除

一部のリソースでは、一部の処理のみがサポートされます。各リソースの詳細については、 ONTAP クラスタで ONTAP API ドキュメントページを参照してください。

オブジェクト **ID**

各リソースインスタンスまたはオブジェクトには、作成時に一意の識別子が割り当てられます。ほとんどの場合、識別子は 128 ビットの UUID です。これらの識別子は、特定の ONTAP クラスタ内でグローバルに一意です。新しいオブジェクトインスタンスを作成する API 呼び出しを実行すると、関連付けられた ID の値を含む URL が HTTP 応答の場所ヘッダーにある呼び出し元に返されます。リソースインスタンスを以降の呼び出しで参照する際には、この識別子を抽出して使用できます。

オブジェクト識別子の内容と内部構造は、いつでも変更される可能性があります。識別子を使用するのは、該当する API 呼び出しで関連付けられているオブジェクトを参照するときに必要に応じてのみです。

オブジェクトのインスタンスとコレクション

リソースパスと HTTP メソッドに応じて、 API 呼び出しを特定のオブジェクトインスタンスまたはオブジェクトのコレクションに適用できます。

同期操作と非同期操作

ONTAP では、クライアントから受信した HTTP 要求を 2 つの方法で実行します。

同期処理

GET 、 HEAD 、 OPTIONS の各メソッドを使用する要求は、いずれも常に同期的に実行されます。また、 POST 、 PATCH 、 DELETE を使用する要求は、完了までに 2 秒かからないと予想される場合は同期的に実行するように設計されています。

非同期処理

POST 、 PATCH 、 DELETE の各メソッドを使用する要求は、完了までに 2 秒以上かかると予想される場合は非同期で実行するように設計されています。

「 return _timeout 」クエリパラメータは、非同期 API 呼び出しで使用でき、非同期呼び出しを同期的に完了するように変換できます。詳細については ' ジョブオブジェクトを使用した非同期処理の動作を参照してください

セキュリティ

REST API のセキュリティは、主に ONTAP で利用可能な既存のセキュリティ機能に基づいています。API で使用されるセキュリティは次のとおりです。

- トランスポートレイヤのセキュリティ

  ONTAP LIF とクライアントの間でネットワークを介して送信されるトラフィックは、通常、 ONTAP の設定に基づいて TLS を使用してすべて暗号化されます。

- クライアント認証

  ONTAP System Manager および Network Manageability SDK と同じ認証オプションを ONTAP REST API でも使用できます。

- HTTP 認証

  HTTP レベルでは、 API トランザクションにベーシック認証が使用されます。base64 文字列のユーザ名とパスワードを含む HTTP ヘッダーが各要求に追加されます。

- ONTAP 許可

  ONTAP では、ロールベースの認証モデルが実装されています。ONTAP REST API または API ドキュメントページにアクセスするときは、適切な権限を持つアカウントを使用する必要があります。

"RBAC を使用したセキュリティ"

API 要求を制御する入力変数

API 呼び出しの処理方法は、 HTTP 要求で設定されたパラメータと変数を使用して制御できます。

HTTP メソッド

次の表に、 ONTAP REST API でサポートされる HTTP メソッドを示します。

REST エンドポイントのそれぞれですべての HTTP メソッドを使用できるわけではありません。また、 PATCH と DELETE はどちらもコレクションで使用できます。詳細については、「 _ オブジェクト参照と Access_ 」を参照してください。

HTTP メソッド	説明
取得	リソースインスタンスまたはコレクションのオブジェクトプロパティを取得します。
投稿（ Post ）	指定した入力に基づいて新しいリソースインスタンスを作成します。
パッチ	指定した入力に基づいて既存のリソースインスタンスを更新します。
削除	既存のリソースインスタンスを削除します。
頭部	実質的には GET 要求を発行しますが、 HTTP ヘッダーのみを返します。
オプション（ Options ）	特定のエンドポイントでサポートされる HTTP メソッドを確認します。

要求ヘッダー

HTTP 要求には複数のヘッダーを含める必要があります。

コンテンツタイプ

要求の本文に JSON が含まれている場合は、このヘッダーを「 application/json 」に設定する必要があります。

同意します

このヘッダーは 'application/hal+json に設定する必要があります代わりに application/json' に設定した場合 ' 次のレコードのバッチを取得するために必要なリンクを除き 'HAL リンクは返されませんヘッダがこれらの 2 つの値以外のものである場合、応答の「 content-type 」ヘッダのデフォルト値は「 application/hal+json 」になります。

承認

base64 文字列としてエンコードされたユーザ名とパスワードを使用するベーシック認証を設定する必要があります。

本文を要求します

要求の本文の内容は、それぞれの呼び出しに応じて異なります。HTTP 要求の本文は、次のいずれかで構成されます。

- JSON オブジェクトと入力変数
- JSON オブジェクトが空です

オブジェクトのフィルタリング

GET を使用する API 呼び出しを発行する際、返されるオブジェクトを任意の属性に基づいて制限またはフィルタできます。たとえば、一致する正確な値を指定できます。

< フィールド >=< クエリ値 >

完全一致に加えて、他の演算子を使用して、一連のオブジェクトを一定範囲の値で返すことができます。次の表に、 ONTAP REST API でサポートされるフィルタ演算子を示します。

演算子	説明
=	等しい
<	より小さい
>	が次の値より大きい
⇐	が次の値以下です
>=	が次の値以上である必要があります
更新	または
！	と等しくない
*	すべてに一致するワイルドカード

また 'null' キーワードまたはその negation!null' をクエリの一部として使用して ' 特定のフィールドが設定されているかどうかに基づいてオブジェクトのコレクションを返すこともできます

通常、設定されていないフィールドはクエリの照合から除外されます。

特定のオブジェクトフィールドを要求しています

デフォルトでは、 GET を使用する API 呼び出しを発行すると、オブジェクトを一意に識別する属性のみが HAL 自己リンクとともに返されます。この最小のフィールドセットは、各オブジェクトのキーとして機能し、オブジェクトタイプによって異なります。次の方法で 'fields クエリー・パラメータを使用して ' 追加のオブジェクト・プロパティを選択できます

- 共通または標準のフィールド

  最もよく使用されるオブジェクトフィールドを取得するには 'fields=*`` を指定しますこれらのフィールドは、通常、ローカルサーバメモリに保持されるか、ほとんど処理を必要としません。これらのプロパティは、 URL パスキー（ UUID ）を指定して GET を使用した場合にオブジェクトに対して返されるプロパティと同じです。

- すべてのフィールド

  「 fields=** 」を指定すると、アクセスするために追加のサーバ処理が必要なフィールドも含め、すべてのオブジェクトフィールドが取得されます。

- カスタムフィールドの選択

  希望するフィールドを指定するには 'field=<field_name>` を使用します複数のフィールドを要求する場合は、スペースを入れずにカンマで区切る必要があります。

> ベストプラクティスとして、必要なフィールドを常に個別に指定することを推奨します。一連の共通フィールドまたはすべてのフィールドを取得するのは、必要な場合だけにしてください。共通として分類され 'fields =*' を使用して返されるフィールドは ' 内部パフォーマンス分析に基づいてネットアップが決定しますフィールドの分類は、今後のリリースで変更される可能性があります。

出力セット内のオブジェクトのソート

リソースコレクション内のレコードは、オブジェクトによって定義されたデフォルトの順序で返されます。フィールド名とソート方向を指定して 'ORDER BY クエリー・パラメータを使用すると ' 順序を次のように変更できます

`order_by=<` フィールド名 `>asc|desc`

たとえば、タイプフィールドを降順でソートし、 ID を昇順でソートできます。

```
order_by=type desc, id asc
```

次の点に注意してください。

- ソートフィールドを指定しても方向を指定しない場合、値は昇順でソートされます。
- 複数のパラメータを指定する場合は、各フィールドをカンマで区切る必要があります。

オブジェクトのコレクションを取得するときのページ付けです

GET を使用する API 呼び出しを発行して同じタイプのオブジェクトのコレクションにアクセスする場合、ONTAP では 2 つの制約に基づいて可能なかぎり多くのオブジェクトを返します。これらの各制約は、要求に対する追加のクエリパラメータを使用して制御できます。特定の GET 要求に対する最初の制約に達した時点で要求が終了されるため、返されるレコードの数が制限されます。

> すべてのオブジェクトについての処理が完了する前に要求が終了した場合、次のレコードのバッチを取得するために必要なリンクが応答に含まれます。

オブジェクト数の制限

「 mAX_records =20`

実際に返されるオブジェクトの数は、関連する時間の制約やシステム内のオブジェクトの総数に基づいて、有効な最大数よりも少なくなることがあります。

オブジェクトを読み出す時間を制限しています

```
re turn _timeout =5
```

実際に返されるオブジェクトの数は、関連するオブジェクト数の制約やシステム内のオブジェクトの総数に基づいて、有効な最大数よりも少なくなることがあります。

結果セットの絞り込み

```
time⇒ 2018-04-04T15:41:29.140265Z & max_records =10
```

複数の要求を問題で処理して、オブジェクトをページングできます。以降の API 呼び出しでは、前回の結果セットの最新イベントに基づいて新しい時間の値を使用する必要があります。

サイズのプロパティ

一部の API 呼び出しおよびクエリパラメータでは、入力値として数値が使用されます。バイト単位で整数を指定する代わりに、必要に応じて次の表に示すサフィックスを使用できます。

サフィックス	説明
KB	KB キロバイト（ 1024 バイト）またはキビバイト
MB	MB （ KB x 1024 バイト）またはメビバイト
GB	ギガバイト（ MB x 1024 バイト）またはギビバイト
容量	TB （ GB x 1024 バイト）またはテビバイト
PB	PB ペタバイト（ TB x 1024 バイト）またはペビバイト

"オブジェクトの参照とアクセス"

API 応答の解釈

各 API 要求でクライアントへの応答が生成されます。応答を調べて成功したかどうかを確認し、必要に応じて追加データを取得します。

HTTP ステータスコード

ONTAP REST API で使用される HTTP ステータスコードを次に示します。

コード	理由フレーズ	説明
200	わかりました	新しいオブジェクトを作成しない呼び出しが成功したことを示します。
201	作成済み	オブジェクトが作成されました。応答の location ヘッダーにオブジェクトの一意の識別子が含まれます。
202.	承認済み	バックグラウンドジョブで要求の実行が開始されましたが、まだ完了していません。
400	無効な要求です	要求の入力が認識されないか不適切です。
401	権限がありません	ユーザ認証に失敗しました。
403	禁止されている	認証（ RBAC ）エラーによりアクセスが拒否されました。
404	が見つかりません	要求で参照されているリソースが存在しません。
405	メソッドを使用できません	要求の HTTP メソッドがリソースに対してサポートされていません。

コード	理由フレーズ	説明
409	競合しています	先に別のオブジェクトを作成する必要があるか、要求されたオブジェクトがすでに存在するため、オブジェクトの作成に失敗しました。
500	内部エラー	サーバで一般的な内部エラーが発生しました。

応答ヘッダー

ONTAP によって生成される HTTP 応答には、いくつかのヘッダーが含まれています。

場所

オブジェクトが作成されると、オブジェクトに割り当てられた一意の識別子を含む、新しいオブジェクトへの完全な URL が location ヘッダーに含まれます。

コンテンツタイプ

通常は 'application/hal+json になります

応答の本文

API 要求の結果として返される応答の本文の内容は、オブジェクト、処理タイプ、および要求の成功または失敗によって異なります。応答は常に JSON 形式になります。

- 単一のオブジェクト

  1 つのオブジェクトを要求に基づいて一連のフィールドとともに返すことができます。たとえば、 GET では、一意の識別子を使用してクラスタの選択したプロパティを取得できます。

- 複数のオブジェクト

  リソースコレクションから複数のオブジェクトを返すことができます。いずれの場合も ' 一貫性のある形式が使用されており 'num_records にはオブジェクト・インスタンスの配列を含むレコードおよびレコードの数が示されますたとえば、特定のクラスタで定義されているノードを取得できます。

- ジョブオブジェクト

  API 呼び出しが非同期で処理されると、バックグラウンドタスクのアンカーを設定するジョブオブジェクトが返されます。たとえば、クラスタ構成の更新に使用される PATCH 要求は非同期で処理され、ジョブオブジェクトが返されます。

- エラーオブジェクト

  エラーが発生した場合は、常にエラーオブジェクトが返されます。たとえば、クラスタに定義されていないフィールドを変更しようとするとエラーが表示されます。

- JSON オブジェクトが空です

  場合によっては、データが返されず、応答の本文に空の JSON オブジェクトが含まれることがあります。

HAL リンク

ONTAP REST API では、 Hypermedia as the Engine of Application State （ HATEOAS ）をサポートするメカニズムとして HAL を使用しています。特定のリソースを識別するオブジェクトまたは属性を返す場合、リソースに関する追加の詳細を簡単に見つけて確認できるように HAL 形式のリンクも返されます。

エラー

エラーが発生した場合は、応答の本文でエラーオブジェクトが返されます。

の形式で入力し

```
"error": {
"message": "<string>",
"code": <integer>[,
"target": "<string>"]
}
```

code の値で一般的なエラーの種類やカテゴリを特定し、 message で具体的なエラーの内容を確認できます。該当する場合、エラーに関連する特定のユーザ入力がターゲットフィールドに表示されます。

一般的なエラーコード

コード		説明
1.	409	同じ識別子のオブジェクトがすでに存在します。
2.	400	フィールドの値が無効であるか、値が指定されていないか、余分なフィールドが指定されています。
3.	400	この処理はサポートされません。
4.	405	指定した識別子のオブジェクトが見つかりません。
6.	403	要求を実行する権限が拒否されました。
8.	409	リソースが使用中です。

ジョブオブジェクトを使用した非同期処理の動作

非同期で実行するように設計された API 要求を発行すると、常にジョブオブジェクトが作成されて呼び出し元に返されます。このジョブは、要求を処理するバックグラウンドタスクを記述してアンカーを設定します。HTTP ステータスコードによっては、要求が成功したかどうかを確認するためにジョブの状態を取得する必要があります。

を参照してください "API リファレンス" 非同期で実行するように設計されている API 呼び出しを確認します。

要求の処理方法を制御する

「 return turn _ timeout 」クエリーパラメータを使用して、非同期 API 呼び出しの処理方法を制御できます。このパラメータを使用した場合の結果には次の 2 つがあります。

要求が完了する前にタイマーが期限切れになります

有効な要求の場合、 ONTAP はジョブオブジェクトとともに HTTP ステータスコード 202 を返します。要求が正常に完了したかどうかを確認するには、ジョブの状態を取得する必要があります。

要求は、タイマーが切れる前に完了します

`re turn _timeout' パラメータのデフォルト値は 0 秒ですしたがって、パラメータを指定しない場合は、有効な要求に対して常に HTTP ステータスコード 202 が返されます。

API 要求に関連付けられたジョブオブジェクトの照会

HTTP 応答で返されるジョブオブジェクトには、いくつかのプロパティが含まれています。後続の API 呼び出しで state プロパティを照会することで、要求が正常に完了したかどうかを確認できます。ジョブオブジェクトは常に次のいずれかの状態になります。

非終了状態

- キューに登録され
- 実行中です
- 一時停止中

終了状態

- 成功
- 失敗

非同期要求を発行するための一般的な手順

非同期 API 呼び出しを完了する大まかな手順を次に示します。この例では、「 re turn _timeout 」パラメータが使用されていないか、バックグラウンドジョブが完了する前に時間が経過していると仮定しています。

1. 問題非同期で実行するように設計された API 呼び出し。
2. 有効な要求を受け入れたことを示す HTTP 応答 202 を受信します。
3. 応答の本文からジョブオブジェクトの識別子を抽出します。
4. タイミングループ内で、サイクルごとに次の操作を実行します。
   a. ジョブの現在の状態を取得します。
   b. ジョブが非終了状態の場合は、もう一度ループを実行します。
5. ジョブが終了状態（ success または failure ）になったら停止します。

"ワークフロー 1 ：クラスタの連絡先の更新とジョブの状態の確認"

Copyright Information

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

Trademark Information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.